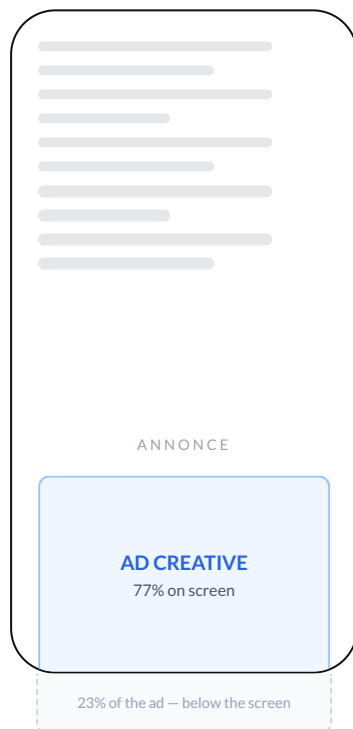


WebAdView SDK

How it works & how to use it



What it does, how it works, and how to get started —
for native iOS apps and Flutter apps on iOS and Android.



Measured honestly

Google sees the real 77%

• 77% visible — the clock is running

seen for 0.42 s of the 1 s required to count

One integration, two platforms

native Swift on iOS, a Kotlin port on Android — same rules, same results

THE SDK, SIMPLY

- 1 What is WebAdView?
- 2 How it works
- 3 An ad's journey
- 4 Consent, simply
- 5 Loading, tuned by STEP Network
- 6 Measured honestly
- 6 The resizing, up close

IMPLEMENTATION GUIDE

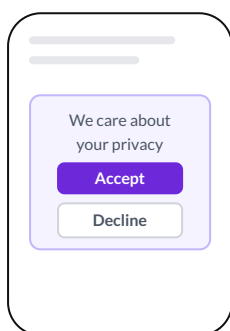
- 7 Install & initialize — native iOS
- 8 Install & initialize — Flutter
- 9 Adding ads to your app
- 10 Using your own consent platform
- 11 Debugging

What is WebAdView?

WebAdView is STEP Network's ad component for apps. A developer places an ad spot in the app the same way they place a picture or a text block — **the SDK takes care of everything else.**

The ads are served by the same ad technology STEP Network uses on the web. That is the core idea: because the ads are web ads, STEP Network can manage sizes, formats, demand and settings **remotely** — things can be adjusted for a publisher without the app being updated.

THE SDK TAKES CARE OF THREE THINGS



1 • Consent first

No ad loads before the user has answered the consent notice — the SDK's own box, or the one the app already has (section 4).



2 • The right moment

Ads load just below the screen as the user scrolls — ready to appear instantly, nothing wasted (section 5).



3 • Really seen

The SDK measures whether the ad was truly on screen — and makes Google's reports show the same truth (section 6).

WHICH INTEGRATION IS FOR YOU

IOS Native iOS app

SwiftUI or UIKit, iOS 16 or newer. Add the Swift Package (section 7).

FLUTTER Flutter app

One Dart integration covers iOS 16+ and Android 7+ (section 8). Underneath, the plugin runs the native SDK on iOS and a Kotlin port of it on Android.

ANDROID Native Android app

Not offered as a standalone library yet — the Kotlin code ships inside the Flutter plugin. Talk to STEP Network.

WHAT A PUBLISHER NEEDS

- 1 An app** — native iOS (SwiftUI/UIKit, iOS 16+) or Flutter 3.44+ targeting iOS 16+ and Android 7+
- 2 Ad unit IDs** — one per ad placement, from the STEP Network onboarding
- 3 A consent key** — unless the app already has its own consent platform (section 4)
- 4 An ad template** — an HTML template page hosted on the publisher's own domain; the ads render inside it, and the SDK is given its URL

And it's light. Ads only load when the user scrolls near them, so the SDK costs nothing while no ad is on screen. On iOS it adds about 8 MB to the app.

How it works

Every ad lives in a small web view — a tiny browser window inside the app. Four layers work together, and each one has a single job.

The app

Decides **where** ads appear in the layout, and which placements to use — nothing more.



The SDK

Handles the rest automatically: waits for consent, loads ads at the right moment, measures whether they are really seen, and opens ad clicks in the browser.



The ad page

A small web page, hosted on the publisher's own website, where the ad is requested and rendered — by the same ad technology STEP Network uses on the web (Yield Manager and Google's ad system).



STEP Network & Google

Everything about the ads themselves — sizes, formats, demand, loading distances — is decided here, remotely. STEP Network can adjust it for any publisher without an app update.

THREE RULES THE SDK NEVER BREAKS

Consent first. Nothing loads before the user has answered the consent notice — and the consent box never appears inside an ad.

Load late. Ads load near the screen, not three screens below it — no wasted requests, no slow feeds.

Measure honestly. Google is shown only the part of the ad that is really on screen, so the reports reflect reality — not an automatic “100% visible”.

ONE SDK, TWO PLATFORMS

IOS The native Swift SDK

Used directly by native iOS apps, and underneath the Flutter plugin on iOS. Ads render in Apple's WKWebView; consent runs through Didomi's iOS SDK.

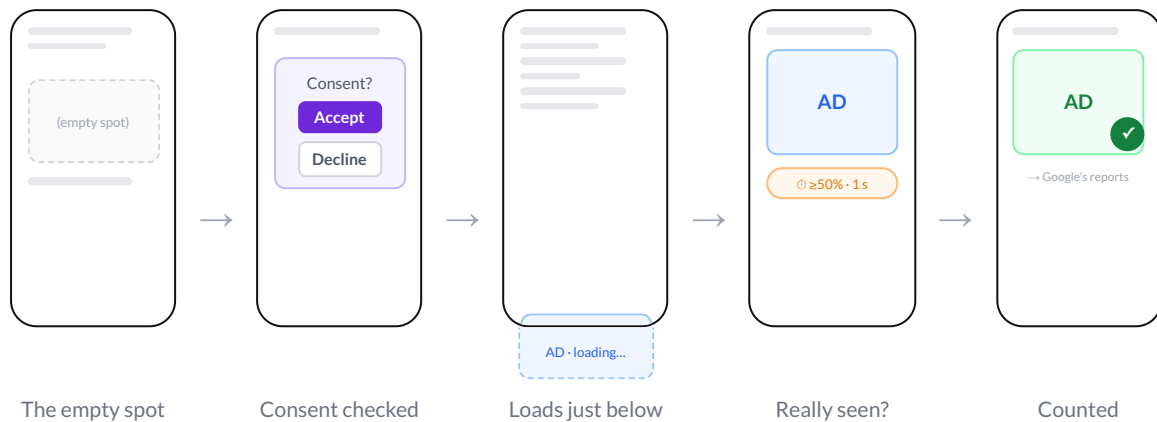
ANDROID A Kotlin port of the same SDK

Used by the Flutter plugin on Android. Ads render in Android's WebView; consent runs through Didomi's Android SDK. Same loading rules, same measurement rules, same log lines.

The two are kept in step by one written contract for the ad page, the same ad template, and the same end-to-end test on both platforms. A publisher's Flutter app never sees the difference.

An ad's journey

From an empty spot in the layout to a counted, viewable ad — in five steps.



1-2 An empty spot — until consent is answered

The placement starts as a quiet placeholder. Has the user answered the consent notice? If not, the ad simply waits — nothing is requested, nothing is sent. The moment an answer exists, waiting ads continue by themselves (section 4).

3 It loads quietly as the user scrolls closer

Typically about 1½ screens away the ad page starts loading in the background, and about one screen away the ad is requested and rendered — the exact distances are set per publisher by STEP Network (section 5). The spot smoothly takes the size of the ad that arrived.

4 The SDK checks that it is really seen

Viewable = at least half the ad on screen for one full second, uninterrupted — the official industry standard. Scrolling away too soon, or leaving the app, restarts the clock (section 6).

5 Google counts it

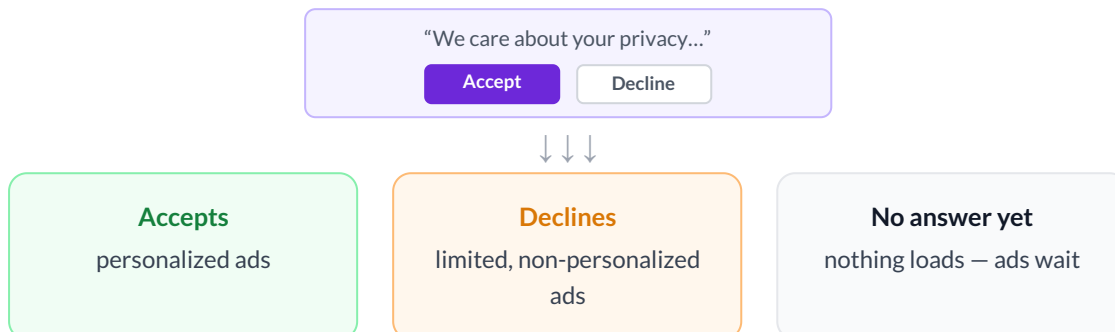
Google runs the same check with its own measurement — which the SDK has made honest — and the viewable impression lands in Google Ad Manager's reports (section 6).

What if the user never scrolls down? Then the ad is never requested at all — no wasted impressions, and no money spent on ads nobody could have seen.

What if the user changes their consent later? Ads already on screen notice by themselves and reload with the new answer. The app doesn't have to do anything.

Consent, simply

One rule: **no ad loads before the user has answered the consent notice.** Both answers open the gate — what's flexible is where the answer comes from.



WHERE THE ANSWER CAN COME FROM — THE THREE MODES

1 • The SDK brings the consent box

For apps that don't have one. The SDK shows the notice (Didomi — the platform STEP Network uses), remembers the answer, and passes it to every ad. The app just provides its key. On Android the same Didomi notice is shown by Didomi's Android SDK.

2 • The app already runs Didomi

The app uses Didomi itself. The SDK shows no second consent box — it asks the app's Didomi directly “has the user answered yet?”, so ads can never start before the notice is answered (setup in section 10).

3 • The app uses another consent system

Any other standard (TCF) consent platform keeps working exactly as today — the SDK shows nothing and just reads its answer, from the standard place the platform stores it on each OS (setup in section 10). *Verified so far with Cookiebot in the iOS simulator and with a simulated TCF platform on iOS and Android — see section 10.*

GOOD TO KNOW

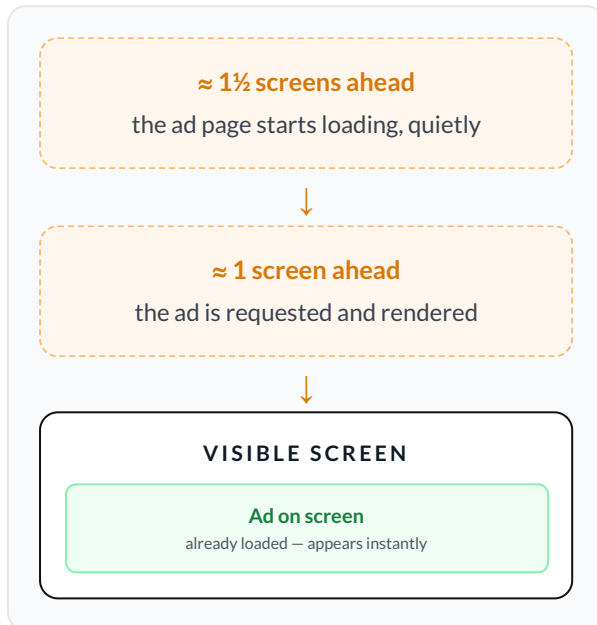
The consent box never appears inside an ad. The answer is collected once, in the app, and handed to every ad page invisibly.

Changing your mind works instantly. If the user opens privacy settings and changes their answer, waiting ads load and ads already on screen reload themselves — automatically.

Consent is checked in two independent places. First, the SDK never even starts loading an ad before the user has answered. Second, the ad technology inside the ad page (Yield Manager and Google's ad system) checks the answer again by itself — exactly as it does on normal websites. If one of the two checks ever failed, the other would still protect the user.

Loading, tuned by STEP Network

Ads load in two steps as the user scrolls toward them — and how early each step happens is a dial STEP Network controls remotely, per publisher.

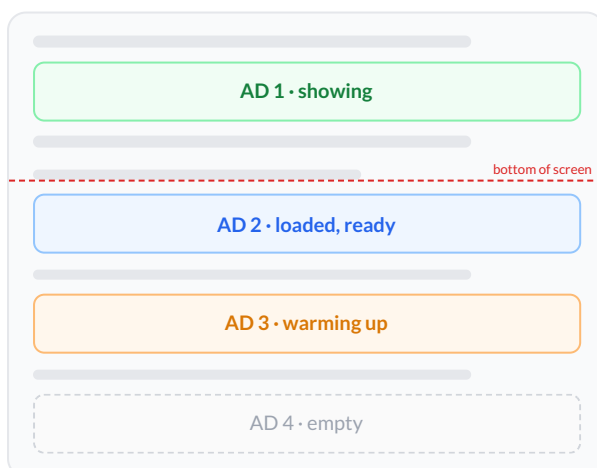


Why two steps? Loading early means the ad is warm and ready — it appears with no visible loading. Showing it late means the impression happens where the user is actually looking — better viewability, and no requests for ads nobody scrolls to.

The distances come from STEP Network — as percentages of the screen. Each publisher gets its own values, for example `fetch: 150` and `render: 100`. The SDK measures the user's actual screen and converts: on a screen 800 points tall, 150% becomes 1,200 points — that is the “1.5 screens”. Percentages give the same experience on every device, big or small. They can be adjusted live at any time — **no app update needed** — and the app remembers them between launches.

The numbers in the app are only a starting point. Whatever a developer might set in code, STEP Network's remote values always take over. That keeps the tuning in one place, for every publisher — on iOS and Android alike.

WHICH ADS ARE LOADED — AND WHICH CAN COUNT



- **On screen — showing and being measured.** Only here can the ad become a viewable impression: at least half visible, for one second (section 6).
- **Just below — the ad is there, ready to appear instantly.** But off screen its window is a 1-point sliver, so it measures as not visible — **it cannot count as viewable down here.**
- **Further down — only warming up.** The ad page is loading quietly in the background; **no ad has been requested yet.**
- **Far away — nothing at all.** Not loaded, not requested, not counted. If the user never scrolls here, this ad never happens.

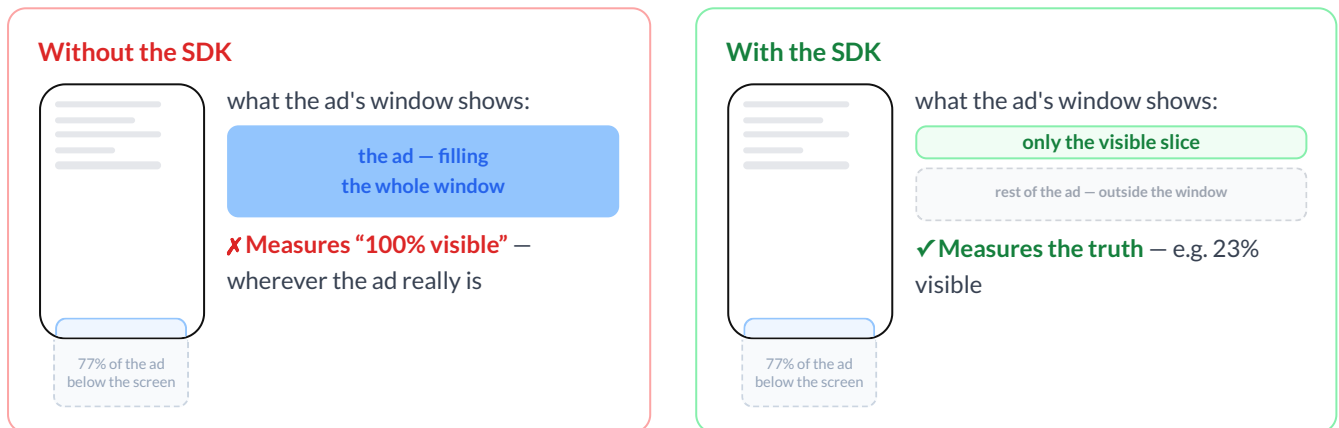
So an ad can never rack up impressions just by existing in the layout — it isn't requested until the user scrolls toward it, and it can only *count* while actually on screen. **And refreshing?** The SDK never swaps an ad on a timer; interval refresh is a Yield Manager feature STEP Network configures remotely, inside the ad page.

Measured honestly

An ad counts as **viewable** when it meets the official industry standard (IAB/MRC) — the one advertisers audit against. The SDK makes sure that standard is applied truthfully in an app.



Why this was a big issue. Every ad sits in its own little browser window, and Google measures from *inside* that window — it can only see the window, never the phone around it. The ad always fills its own window, so Google always said “**100% visible**”, no matter where the ad really was. Numbers nobody could trust.



HOW THE SDK DOES IT — STEP BY STEP

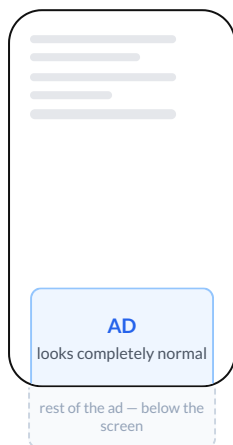
- 1 It always knows what's visible.**
The SDK tracks the ad's position natively, continuously — at every moment it knows exactly which part of the ad is on screen. The clock only starts once a creative has actually rendered; an empty or still-loading slot never counts.
- 2 It makes the ad's window match.**
The window becomes exactly as big as the visible part of the ad — the part below the screen simply has no window anymore. For the user, the ad looks exactly the same.
- 3 Google now measures the truth.**
Its code still just measures the window — but the window now equals what is really on screen, so “% visible” and viewable impressions become real.
- 4 And it never disturbs anything.**
The resizing only begins once the ad is actually showing, and the ad's spot in the app keeps its size the whole time — the user never sees flickering or jumping content. Identical on iOS and Android.

One ad counts once. A served ad can become viewable at most once — if the user scrolls away and back to the *same* ad, it is not counted again (that would inflate the numbers). The spot earns a new viewable impression in Google's reports only when a *new* ad is served in it — for example a rebuilt list row, or an ad refresh configured remotely by STEP Network.

The resizing, up close

The same moment, seen from two sides: what the user sees on the phone — and what the SDK has actually built inside the ad's spot.

WHAT THE USER SEES



The ad simply sticks out below the screen — nothing looks special.

WHAT THE SDK HAS ACTUALLY BUILT THERE



The ad's window (webview)

resized to exactly the visible part — this is all Google's measurement can see

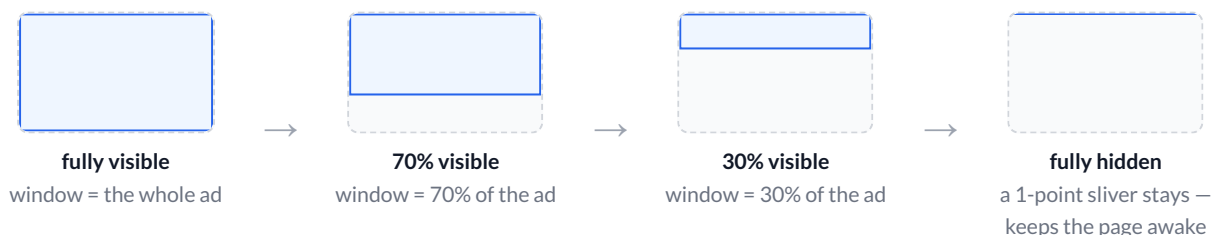
The page inside is shifted

by the same amount the window shrank — so the ad's pixels stay exactly in place on screen

The ad's spot (dashed)

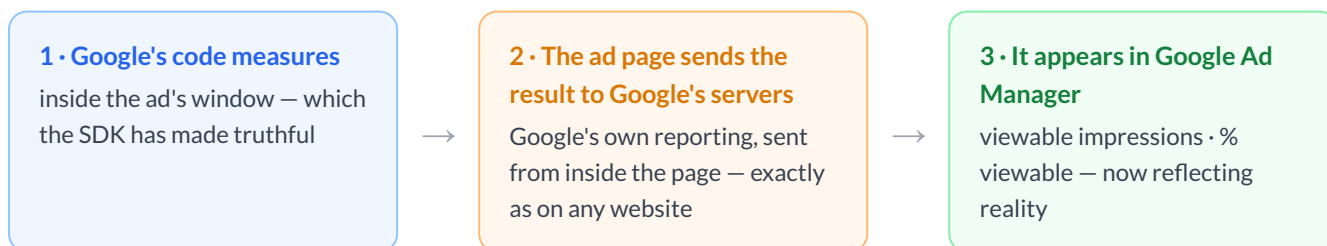
keeps its full size in the layout — that's why nothing ever jumps or shifts for the user

AS THE USER SCROLLS, THE WINDOW FOLLOWS — UPDATED CONTINUOUSLY



The window is never resized before the ad has rendered — an ad is never requested into a collapsed window. On Android the page's zoom level is locked as well, so the creative always renders at its true size while the window changes.

HOW DOES GOOGLE GET THE NUMBERS?



The SDK is never part of this chain — it cannot send or change numbers (nobody can; Google only trusts its own measurement). It only fixes what Google's code sees in step 1. And that is why the 1-point sliver matters: if the window disappeared completely, the phone would put the ad's page to sleep to save battery — and a sleeping page can't send its results to Google. The sliver keeps it awake.

IMPLEMENTATION GUIDE · **IOS** NATIVE IOS APP

Install & initialize — native iOS

Two steps and the SDK is ready: add the package, then give it your publisher details — your consent key and your template link — once, at app launch. Everything on this page is copy-paste friendly. *Building with Flutter? Skip to section 8.*

1 ADD THE PACKAGE

In your app project in Xcode: **File** → **Add Package Dependencies...**, paste the repository URL, and add the **WebAdViewSDK** product to your app target. Then `import WebAdViewSDK` — everything comes through that one module, and Didomi is pulled in automatically.

Use a standard App project. Swift Playgrounds (`.swiftpm`) projects cannot embed the Didomi framework — they may look fine in the simulator but crash on a real device.

2 INITIALIZE AT APP LAUNCH

Call `initialize(config:)` once, before any ad view exists. In a pure-SwiftUI app the simplest place is your `App` struct's `init()` ; if you already have an `AppDelegate`, put the same call at the top of `didFinishLaunchingWithOptions` .

Both values come from your STEP Network onboarding. Treat the Didomi key as a secret — inject it from a build setting rather than committing it. The template URL is the address of your ad template — the HTML page hosted on your own domain (explained in section 2, *How it works*). If `initialize` is never called, ads simply stay blank — and the SDK prints a loud `[SN] [ERROR]` telling you why.

UIKit app? The same package works without SwiftUI: `WebAdScope` owns the loading and measurement for one scrolling screen, and `WebAdHostView` is the ad view you place in your layout. A complete example is in **GUIDE.md**, section 4b.

THE CODE

```
// In your App struct — runs once at launch
import SwiftUI
import WebAdViewSDK

@main
struct MyApp: App {
    init() {
        WebAdViewSDK.initialize(config:
WebAdViewSDKConfig(
    didomiAPIKey: "<YOUR-DIDOMI-API-KEY>",
    adTemplateURL: URL(string:
        "<YOUR-AD-TEMPLATE-URL>")!
    ))
    }

    var body: some Scene {
        WindowGroup { ContentView() }
    }
}
```

NOT USING DIDOMI? (MODES 2-3 IN SECTION 4)

```
// Your app has its own consent platform:
// a consentProvider replaces the Didomi key
WebAdViewSDK.initialize(config: WebAdViewSDKConfig(
    consentProvider: TCFConsentProvider(),
    adTemplateURL: URL(string:
        "<YOUR-AD-TEMPLATE-URL>")!
    ))

// Is your own platform Didomi? Use this instead:
// consentProvider: AppDidomiConsentProvider()
```

Everything else stays the same — details and the template requirement in section 10.

No setup from STEP Network yet? Test ad units and the test template URL are listed in **QUICKSTART.md** in the repository (the test Didomi key is in the demo app), so development can start before the real setup exists.

IMPLEMENTATION GUIDE · FLUTTER · IOS + ANDROID

Install & initialize — Flutter

One Dart package, `webadview_flutter`, gives a Flutter app STEP Network ads on iOS and Android. It runs the native SDK underneath, so everything in sections 1–6 applies unchanged on both platforms.

1 ADD THE PACKAGE

Add `webadview_flutter` to your `pubspec.yaml` as a Git dependency — STEP Network gives you the repository URL and version at onboarding; the plugin lives in the repository's

`Flutter/webadview_flutter` folder. Flutter 3.44 or newer; iOS builds use Swift Package Manager only (CocoaPods is not supported).

iOS: deployment target 16.0. Open `ios/Runner.xcodeproj` and set *Minimum Deployments* to iOS 16 (the Flutter template defaults to 13). The plugin registers itself; nothing else changes in Xcode.

Android: MainActivity must extend `FlutterFragmentActivity`. Didomi's consent notice needs a fragment host. Android 7 (API 24) or newer; Didomi for Android is pulled in automatically.

2 INITIALIZE AT APP LAUNCH

Call `WebAdViewSdk.initialize` once in `main()`, before `runApp`. The two values are the same as on native iOS: your Didomi key (inject it with `--dart-define`, never commit it) and your ad template URL. On web or desktop it logs once and ads render as an empty placeholder.

THE CODE

```
// main.dart — runs once at launch
import 'package:webadview_flutter/webadview_flutter.dart'

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await WebAdViewSdk.initialize(
    WebAdViewSdkConfig.didomi(
      apiKey: const String.fromEnvironment(
        'SN_DIDOMI_API_KEY'),
      adTemplateUrl:
        Uri.parse('<YOUR-AD-TEMPLATE-URL>'),
    ),
  );
  runApp(const MyApp());
}
```

ANDROID: THE ONE-LINE ACTIVITY CHANGE

```
// android/app/src/main/kotlin/.../MainActivity.kt
import io.flutter.embedding.android
    .FlutterFragmentActivity

class MainActivity : FlutterFragmentActivity()
```

NOT USING DIDOMI? (MODES 2-3 IN SECTION 4)

```
// Any TCF-certified consent platform:
await WebAdViewSdk.initialize(
  WebAdViewSdkConfig.tcf(
    adTemplateUrl:
      Uri.parse('<YOUR-AD-TEMPLATE-URL>'),
  ),
);

// Your app runs its own Didomi
// (e.g. the didomi_sdk plugin)? Use:
// WebAdViewSdkConfig.appDidomi(...)
```

Details and the template requirement in section 10. Test ad units and template URL for development before onboarding: **QUICKSTART.md**; the test Didomi key is in the demo app.

Adding ads to your app

Three small additions to a screen you already have — that is the whole integration, on every platform.

IOS SWIFTUI — IN ONE OF YOUR EXISTING SCREENS

```
ScrollView {
  VStack(spacing: 24) {
    // ...your existing content...

    WebAdView(adUnitId: "div-gpt-ad-mobile_1")
      .showAdLabel(true, text: "announce")
      .frame(maxWidth: .infinity)

    // ...more of your content...

    WebAdView(adUnitId: "div-gpt-ad-mobile_2")
      .showAdLabel(true, text: "announce")
      .frame(maxWidth: .infinity)
  }
}
.lazyLoadAd() // REQUIRED
.background(DidomiWrapper()) // consent box
```

FLUTTER DART — THE SAME SCREEN

```
LazyLoadAdScope( // REQUIRED
  child: SingleChildScrollView(
    child: Column(children: [
      // ...your existing content...
      const WebAdView(
        adUnitId: 'div-gpt-ad-mobile_1',
        showAdLabel: true),
      // ...more of your content...
      const WebAdView(
        adUnitId: 'div-gpt-ad-mobile_2',
        showAdLabel: true),
    ]),
),
)
```

One more thing: the consent box only appears once, but users must be able to change their answer later. Add a “Privacy settings” button in your app's settings — calling `WebAdViewSDK.showConsentPreferences()` (Swift) or `WebAdViewSdk.showConsentPreferences` (Dart) reopens the consent choices. In mode 2 the same call opens your Didomi preferences; in mode 3 your own consent platform's settings screen does this job instead.

THE THREE THINGS TO INCLUDE

A `WebAdView` per placement. Placed in your layout like any other view or widget — one per ad unit ID from your onboarding, with flexible width. It takes the size of the ad that arrives.

`.lazyLoadAd()` / `LazyLoadAdScope` on the scroll container — required, not optional. It installs the loading and measurement machinery. Without it, ads stay blank forever. One per scrolling screen. Works with `ScrollView` and `List`, and with `SingleChildScrollView` and `ListView` alike; rows a list rebuilds as the user scrolls back and forth show a fresh ad each time they return — every ad still counts only while actually seen (section 6).

`.background(DidomiWrapper())` once, anywhere in the hierarchy (SwiftUI only). It gives the consent box a place to appear. In Flutter the plugin presents the notice by itself. Only in mode 1 — apps with their own consent setup skip this (section 10).

WHEN YOU RUN YOUR APP

1 The consent notice, on first launch
until it's answered, no ad requests happen at all

2 Answer → the first ad loads
within a couple of seconds

3 Scroll → further ads load near the screen
and can only count as viewable while actually seen

4 Tagged lines in the console
with debug on (section 11): loading, consent and measurement, live

Optional extras — custom targeting, sizing hints and viewability callbacks — are documented in [GUIDE.md](#) (native iOS) and the plugin's [README.md](#) (Flutter) in the repository.

Using your own consent platform

Does your app already show its own consent box? Then the SDK must not show a second one — it reads your platform's answer instead, and your consent box keeps working exactly as today.

THE ONLY CODE CHANGE

```
// Swift (native iOS)
WebAdViewSDK.initialize(
  config: WebAdViewSDKConfig(
    consentProvider: TCFConsentProvider(),
    adTemplateURL: URL(string:
      "<YOUR-AD-TEMPLATE-URL>")!
  )
)
```

```
// Dart (Flutter, iOS + Android)
await WebAdViewSdk.initialize(
  WebAdViewSdkConfig.tcf(
    adTemplateUrl:
      Uri.parse('<YOUR-AD-TEMPLATE-URL>'),
  ),
);
```

No Didomi key, and no `DidomiWrapper()` in your views — your platform owns the consent UI. Everything else from section 9 stays the same. The SDK reads the answer from the standard place a TCF platform stores it: `UserDefaults` on iOS, `SharedPreferences` on Android.

WHAT CHANGES — AND WHAT DOESN'T

Your consent box: unchanged

your platform shows the notice and the preferences screen, exactly as today

The gate: unchanged

ads wait until your platform has an answer — and reload by themselves if the user changes it later (section 4)

The ad template: one requirement

the template page must contain **no consent code of its own** — tell STEP Network you use your own platform and they provide a matching template

Is your own platform Didomi? (mode 2) Use

`consentProvider: AppDidomiConsentProvider()` in Swift, or `WebAdViewSdkConfig.appDidomi(...)` in Dart — the SDK then asks your Didomi directly whether the notice has been answered. Add Didomi through the same package source the SDK uses, so both share one copy.

ONE CHECK BEFORE LAUNCH

Ads must wait for the notice. Some consent platforms write their consent value *before* the user has answered — then ads could start too early. The check: delete the app, reinstall, and don't touch the notice — **no ads may load until you answer**. If they do, your platform's dashboard has a setting to delay the consent value until the user has interacted. (Mode 2, app-owned Didomi, is immune — the SDK asks Didomi directly.)

What has been verified so far: Cookiebot (Usercentrics, TCF 2.2) in the iOS simulator; and, on both iOS and Android, a simulated TCF platform plus the app-owned Didomi variant — each proving that ads wait for the answer, load on it and reload when it changes. Other platforms use the same mechanism but haven't been tested yet — every publisher's setup is validated together with STEP Network before launch.

IMPLEMENTATION GUIDE

Debugging

One switch turns on readable console logs, so you can watch loading, consent and measurement happen live. It's off by default — release builds stay clean automatically.

IOS NATIVE

```
DebugSettings.shared.isDebugEnabled = true
```

Then watch Xcode's console, or from a terminal:

```
xcrun simctl launch --console-pty \
  booted <bundle-id> | grep '\[SN\]'
```

FLUTTER IOS + ANDROID

```
await WebAdViewSdk.setDebugEnabled(true);
```

iOS: same console command as on the left. Android (the Dart-side `[SN]` `[FLUTTER]` lines appear under logcat's `flutter` tag — drop `-s SN` and grep `'\[SN\]'` to see everything):

```
adb logcat -s SN | grep VIEWABILITY
```

Set it anywhere — it works immediately, no rebuild needed, and it is remembered between launches. The demo apps' ladybug button is exactly this switch.

HOW TO READ THE LOG TAGS

`[SN]` `[LLM]` — loading: each ad's state changes as the user scrolls (`notLoaded` → `fetches` → `displayed`).

`[SN]` `[VIEWABILITY]` — measurement: % visible and the timer, ending in  **VIEWABLE** when an impression counts; `[ACTIVEVIEW]` lines are Google confirming the same impression.

`[SN]` `[CLIP]` — the honest-measurement resizing: the window size the ad page currently gets (section 6).

`[SN]` `[NATIVE]` — consent and setup: readiness, answers, holdbacks and reloads.

`[SN]` `[FLUTTER]` — the bridge between Dart and native, Flutter apps only.

`[SN]` `[WebAdView]` `[HTML]` — messages from inside the ad page, bridged out so you never need a web inspector.

WHAT A HEALTHY SCROLL LOOKS LIKE

```
[SN] [LLM] Ad div-gpt-ad-mobile_2 TRANSITION: notLoaded → fetched
[SN] [LLM] Ad div-gpt-ad-mobile_2 TRANSITION: fetched → displayed
[SN] [VIEWABILITY] div-gpt-ad-mobile_2: 52% visible - ≥50% reached, continuous timer STARTED (needs 1.0s, display)
[SN] [VIEWABILITY] div-gpt-ad-mobile_2: 93% visible | in-view 1.00s / 1.00s (display) |  VIEWABLE (latched)
[SN] [VIEWABILITY] [ACTIVEVIEW] div-gpt-ad-mobile_2__...: impressionViewable - Google counted a viewable impression
```

The same five lines appear on iOS and Android — the two implementations share their log format on purpose, so STEP Network can validate either platform the same way.